Lerna Protocol

Claim-UTxO Settlement for Hydra/Cardano eUTxO

Arthur Zhang

6 June 2026

Abstract

Lerna is an implementation-backed claim-UTxO settlement discipline for Hydra-family Cardano eUTxO systems. It replaces a prose-only factory exit story with a concrete claim object, an Aiken spending validator, deterministic off-chain transition checks, real Hydra/Cardano devnet evidence, and an executable security-theorem gate. The implemented claim binds each settlement to one domain, network id, Hydra Head id, factory id, and claim epoch; requires strict latest-state supersession; conserves lovelace and native assets across selected settlement outputs and continuation state; emits release receipts for materialised rights; supports reduced-signature non-interference, native-asset, and multi-batch profiles; and rejects production acceptance unless local ledger artifacts hash-match the recorded evidence.

The claim is intentionally bounded. Lerna is not a mainnet deployment package, not a replacement for Hydra close/contest/fanout, and not a proof that every virtual-channel application can settle within arbitrary transaction budgets. The theorem checked by the repository is conditional on Hydra contestability, Cardano ledger validation of submitted transactions, proof availability, watcher/operator liveness, and materialisation capacity inside the committed claim budget. Under those assumptions, the current implementation makes the paper's security surface auditable as code: the validator, deterministic model, real devnet verifier, profile matrix, local file-hash checks, and negative tests must all cover the same obligations.

Introduction

Hydra Head gives a fixed participant set a Cardano-like off-chain ledger: participants deposit UTxOs, sign snapshots, close with a snapshot when necessary, contest stale closes, and fan out the resolved UTxO set to L1 [hydra-paper,hydra-docs]. This is already a powerful settlement substrate. A virtual-channel factory layered inside or near a Head should therefore avoid inventing an unmeasured parallel dispute game. Its hard problem is narrower and more operational: when compact rights are no longer being updated optimistically, what exact ledger object can materialise them without double release, stale-state settlement, or silent asset loss?

The current Lerna implementation answers with a script-controlled claim UTxO. A claim UTxO is a compact ledger object carrying unresolved value, latest-state metadata, authorisation policy, release-receipt state, proof-availability commitments, and materialisation bounds. Settlement spends the claim through a compiled Aiken validator. A final settlement consumes all unresolved claim value. A partial settlement creates a continuation claim UTxO with an inline datum and a strictly advanced settled-right frontier.

This paper is written to match that implementation. It no longer presents the claim path as a later research branch, nor does it treat Head-local accounting alone as the production target. Historical factory-local ideas remain useful as motivation, but the auditable protocol claim is now the claim-UTxO path checked by the repository at commit 66ff116. The acceptance commands are:

```
npm test
npm run verify:security-theorem
npm run check:devnet
npm run check:profiles
```

The paper and implementation make four concrete contributions.

C1. Claim-UTxO state object.: Lerna defines a claim datum that binds factory identity, Hydra Head identity, network id, claim epoch, latest-state number, unresolved value, signer policy, release-receipt root, proof-availability root, and materialisation bounds.

C2. Ledger-enforced settlement transition.: The Aiken validator checks identity, domain separation, latest-state supersession, continuation shape, release-receipt progress, authorisation, output descriptors, selected output indices, exact value conservation, min-ADA, and continuation/finality conditions.

C3. Profile-backed non-interference and conservation.: The deterministic model and devnet profile matrix cover reduced-signature non-interference, native-asset conservation, and multi-batch continuation. These are not prose-only extensions: each profile has real devnet evidence and must satisfy the theorem gate.

C4. Executable theorem gate.: The repository states security obligations as code and fails acceptance unless the model, validator source, real devnet verifier, local file hashes, profile matrix, and negative tests jointly cover them.

Layering

Lerna stays inside the Cardano/Hydra settlement vocabulary. Cardano L1 supplies eUTxO accounting, validators, inline datums, reference scripts, native assets, validity intervals, and min-ADA rules [cardano-eutxo]. Hydra supplies the outer Head lifecycle: open, snapshot, close, contest, and fanout. Lerna supplies the inner claim object used when compact virtual-channel rights must be materialised into ordinary ledger outputs.

This layering creates two design constraints. First, the claim UTxO must not pretend to replace Hydra contestability. If a stale Head close is not contested by an actor with the right evidence and authority path, Lerna cannot repair the outer ledger result by prose. Secondly, compactness must be repaid before final acceptance. The production report may pass only when the final claim leaves no unresolved lovelace or native assets.

Eltoo motivates non-punitive latest-state replacement [eltoo]. Channel factories and Perun motivate shared funding and virtual-channel adjudication patterns [factories,perun]. Interhead studies virtual Heads across Heads [interhead]. Hydrozoa studies a Cardano state-channel design that separates execution from custody and uses deterministic rollout for large output sets [hydrozoa-spec]. Lerna takes these as boundary lessons, not as drop-in machinery: the current claim is a Cardano/Hydra claim UTxO with measured evidence.

System and Threat Model

Actors

Head participant.: A Hydra party able to run a Hydra node, sign snapshots, close, and contest.

Virtual participant.: An application endpoint whose right is represented in Lerna claim state. A virtual participant may or may not be a Head participant.

Operator.: An application actor that can coordinate batching, submit transactions, or fund ordinary fees. The operator does not gain authority to redirect claim value.

Watcher.: An actor that observes Head lifecycle events and Lerna evidence, and can cause fresh evidence or settlement transactions to be submitted through an authorised path.

Adversary

The adversary may corrupt participants, operators, watchers, or Head participants; withhold signatures; provide stale latest-state evidence; propose same-number conflicting states; withhold proof bodies; try replay across domains, networks, Heads, factories, or epochs; create outputs below min-ADA; over-release lovelace or native assets; reorder multi-batch settlement; mutate continuation datums; or tamper with local evidence files used by production acceptance.

Assumptions

If the Head is closed with stale state, at least one honest actor can observe the close and contest with newer valid Head evidence before the contestation deadline.

Submitted claim-creation, claim-settlement, and continuation transactions are validated by the Cardano ledger according to the observed devnet rules.

Proof bodies committed by the proof witness root are available to the verifier before materialisation is accepted.

At least one actor with sufficient evidence and authority can submit accepted materialisation plans within the relevant Head and ledger deadlines.

The number of outputs, proof bytes, execution units, and continuation value fit within the budget committed in the claim datum.

Claim State

A claim scope is the tuple \[=(domain,networkId,hydraHeadId, factoryId,claimEpoch). \] Every latest-state header, transition, receipt, proof-availability commitment, and non-interference proof is interpreted only inside this scope.

The implementation uses the following domain separators:

Domain & Purpose\\

Lerna:ClaimUTxO:LatestStateHeader & latest-state and transition binding\\ Lerna:ClaimUTxO:ReleaseReceipt & release receipt nullifiers\\ Lerna:ClaimUTxO:ProofAvailability & proof witness package binding\\ Lerna:ClaimUTxO:NonInterferenceProof & reduced-signature frame proof\\

A claim datum \(C\) contains: \[

C = (&factoryId,hydraHeadId,claimEpoch, networkId,domain,latestStateNumber,\\ &stateCommitment,releaseReceiptRoot, nonInterferenceRoot,authorisationMode,\\ &unresolvedLovelace,unresolvedAssets, minAdaFloor,requiredSigners,\\ &settledRightCount,lastSettledRightId, maxOutputCount,maxProofSizeBytes,\\ &maxExecutionMemory,maxExecutionSteps, proofAvailabilityMode,proofWitnessRoot).

\] The state commitment is recomputed from length-prefixed canonical fields, not accepted as an independent assertion.

The claim value is: \[(C)= lovelace(C.unresolvedLovelace) + C.unresolvedAssets. \]

Settlement Transition

A settlement transition spends the current claim UTxO and selects a canonical batch of rights \(R=(r_1,,r_k)\) with matching settlement outputs \(O=(o_1,,o_k)\). It provides a redeemer \(D\), an authorised signer set \(S\), and either a final zero-claim result or a continuation claim datum \(C'\).

Admission

The transition is admitted only if:

- \(D.nextStateNumber > C.latestStateNumber\). • The domain, network id, Head id, factory id, and claim epoch in the redeemer match the datum. • \(k > 0\) and \(k C.maxOutputCount\). • Every selected output is at or above \(C.minAdaFloor\). • Proof size and script-cost evidence are within the committed bounds. • The batch starts strictly after \(C.lastSettledRightId\) in the canonical settled-right order.

Authorisation

Lerna has two implemented authorisation modes. In full authorisation, the settlement evidence must contain exactly the datum-bound signer set. In reduced-signature mode, the authorised signers must be an exact proper subset and the non-interference witness must show that all materialised outputs belong to signed owners and that the untouched frame is preserved.

The repository profile matrix currently passes reduced-signature evidence on a real devnet run. This matters because reduced-signature settlement is a common source of accidental over-claiming: without a frame proof, an output for an unsigned owner could be smuggled into a partial settlement.

Conservation

Let \(M\) be the value materialised by the selected settlement outputs and \(Remainder\) be the value recorded in the continuation claim, or zero for a final settlement. The validator and deterministic model require: \[(C) = M + Remainder. \] This equality is exact over lovelace and native assets. The native-asset profile is therefore not merely a text extension; it is a profile-matrix run whose real devnet evidence must pass.

Release Receipts

For each materialised right \(r_i\) and selected output \(o_i\), Lerna emits a deterministic receipt: \[_i=(Lerna:ClaimUTxO:ReleaseReceipt, , r_i.rightId, desc(o_i), i). \] The transition advances the receipt root from the previous root to a new root covering the settled batch. A later settlement cannot materialise the same right prefix again because the canonical frontier and receipt root must both advance.

Continuation

If unresolved value remains, exactly one continuing output must return to the claim script address with an inline datum \(C'\). That datum must preserve the scope, signer policy, non-interference root, materialisation budget, proof availability mode, and proof witness root, while advancing the latest-state number, receipt root, settled-right count, last settled right id, unresolved value, and state commitment. If no unresolved value remains, there must be no continuing claim output.

Validator Enforcement

The Aiken validator validators/lerna_claim.ak enforces the settlement rules at the script boundary. Its key checks are:

Check family & Validator responsibility\\

Input and continuation & Resolve the own input, compare its value to claim value, require either no continuation for final settlement or exactly one inline-datum continuation.\\ Identity and domain & Bind datum and redeemer to the same factory, Head, network, epoch, and latest-state domain.\\ Latest state & Require the old commitment to match the datum and the next state number to be strictly greater.\\ State commitment & Recompute the current and next state commitments from canonical fields.\\ Receipt progress & Recompute the expected release-receipt root and require it to advance.\\ Authorisation & Check exact full signers or exact reduced-signature non-interference signers.\\ Conservation & Require input claim value to equal selected outputs plus continuation value across lovelace and native assets.\\ Output descriptors & Bind selected output indices, payment key hashes, lovelace, and asset bundles to the redeemer descriptors.\\ Admission bounds &

Enforce positive numeric bounds, output-count bounds, min-ADA policy, and continuation-value safety.\\

These checks are intentionally redundant with the deterministic off-chain model. The model explains and generates expected transitions; the validator decides whether an on-chain spend is admissible; the real devnet verifier checks that the claimed local evidence refers to the actual generated artifacts.

Implementation Theorem

For any production-accepted claim settlement, if the Hydra/Cardano devnet evidence, local files, profile matrix, and theorem gate all pass, then the settlement:

- spends the script-controlled claim UTxO through the compiled claim validator with matching inline datum, redeemer, and artifacts; • binds latest-state evidence to one domain, network id, Head id, factory id, and claim epoch; • strictly supersedes the previous latest-state number; • preserves recomputable datum, transition, and continuation commitments; • uses exact full authorisation or reduced-signature non-interference; • conserves lovelace and native assets across selected outputs and continuation state; • emits deterministic release receipts and advances the settled-right frontier; • respects committed output-count, proof-size, execution-unit, and min-ADA bounds; • binds proof availability before materialisation; • accepts only hash-matching local ledger, transaction, artifact, script-cost, and Hydra lifecycle evidence; and • reaches final acceptance only when no unresolved claim value remains.

The theorem is not a hand-waved claim. It is encoded in offchain/lerna-security-theorem.mjs as named obligations. The verifier fails unless each obligation is covered by the deterministic model, validator source keywords, strict real devnet checks, local file-hash verification, profile-matrix evidence, and negative tests.

Obligation id & Meaning\\

claim-utxo-ledger-enforcement & Claim spend uses the compiled validator and matching artifacts\\ latest-state-supersession & Same-scope settlement strictly advances state number\\ state-commitment-integrity & Commitments are recomputable from canonical fields\\ exact-authorisation & Full or reduced signer evidence matches the datum policy\\ non-interference-frame & Reduced settlement affects only signed owners\\ asset-conservation & Lovelace and native assets are exactly conserved\\ release-receipt-nullifier & Materialised rights cannot be claimed twice\\ multi-batch-progress & Partial batches advance a canonical frontier\\ materialisation-admission & Output count, proof size, costs, and min-ADA are bounded\\ proof-availability & Witness availability is bound before materialisation\\ ledger-evidence-authenticity & Local files and ledger observations hash-match the report\\ finality-of-accepted-report & Final acceptance leaves no unresolved claim value\\

Evidence and Evaluation

The implementation repository contains deterministic evidence, real Hydra/Cardano devnet evidence, and a profile matrix. Production acceptance uses four commands:

```
npm test
npm run verify:security-theorem
npm run check:devnet
npm run check:profiles
```

npm test runs the Node test suite, including negative tests for stale state numbers, signer mismatches, conservation failures, receipt mismatches, output descriptor mutations, and local evidence mismatches. npm run verify:security-theorem checks the obligation coverage. npm run check:devnet probes the toolchain, builds the Aiken blueprint, runs deterministic claim evidence, runs the real Hydra/Cardano devnet flow when available, refreshes deterministic evidence with the real Head scope, verifies the real evidence, and asserts acceptance. npm run check:profiles requires the reduced-signature, native-asset, and multi-batch profiles to pass real devnet verification.

Profile & Authorisation & Native assets & Batching\\

reduced-signature & reduced non-interference & none & final settlement\\ native-asset & reduced non-interference & exact conservation & final settlement\\ multi-batch & reduced non-interference & none & two batches\\

As of the evidence generated on 6 June 2026, all three profiles report passFailStatus = passed, realDevnetPassFailStatus = passed, and noUnresolvedClaimsAfterSettlement = true.

Limitations

The current result is strong enough to change the paper's center of gravity, but it remains bounded.

- The implementation is local devnet evidence, not a mainnet deployment. • The theorem assumes Cardano ledger validation and Hydra lifecycle observations behave as recorded. • Proof bodies committed by the proof witness root must be available to the verifier. • Watcher/operator liveness remains a real deployment assumption. • Materialisation is safe only within the committed output-count, proof, execution-unit, and min-ADA budget. • Dynamic membership, arbitrary application state, policy-level mint/burn logic, and cross-Head settlement composition are outside the accepted claim.

Conclusion

The implementation changes the right paper. Lerna should now be described by the object that is actually checked. The auditable object is the claim UTxO: a script-controlled settlement state with latest-state binding, exact authorisation or reduced-signature non-interference, release-receipt nullifiers, exact asset conservation, multi-batch continuation, materialisation admission, proof availability, and hash-bound devnet evidence.

The remaining work is not to invent a new roadmap label. It is to keep tightening the theorem boundary: expand independent review of the Aiken validator, reproduce devnet evidence on fresh infrastructure, measure mainnet era limits, and state every deployment assumption at the same precision as the code checks it.

Appendix

Reproducibility Checklist

- Check the implementation commit: 66ff116. • Run npm test. • Run npm run verify:security-theorem. • Run npm run check:devnet. • Run npm run check:profiles. • Inspect evidence/profile-matrix/profile-matrix.json. • Inspect docs/LERNA_SECURITY_THEOREM.md and confirm every obligation id appears in the theorem gate.

S. Chakravarty, M. D. Coretti, M. Fitzi, P. Gazi, P. Kant, A. Kiayias, and A. Russell. Hydra: Fast Isomorphic State Channels. IACR Cryptology ePrint Archive, Report 2020/299. https://eprint.iacr.org/2020/299.pdf.

Cardano Scaling. Hydra Head protocol documentation. https://hydra.family/head-protocol/docs/protocol-overview.

Cardano Scaling. Hydra known issues and limitations. Consulted 6 June 2026. https://hydra.family/head-protocol/docs/known-issues.

Cardano Documentation. The Extended UTxO accounting model. https://docs.cardano.org/about-cardano/learn/eutxo-explainer/.

M. Jourenko, M. Larangeira, and K. Tanaka. Interhead Hydra: Two Heads are Better than One. IACR Cryptology ePrint Archive, Report 2021/1188. https://eprint.iacr.org/2021/1188.pdf.

G. Flerovsky, I. Rodionov, and P. Dragos. Hydrozoa: Lightweight state channels for Cardano. Technical specification working draft, 7 November 2025. https://cardano-hydrozoa.github.io/hydrozoa/hydrozoa.pdf.

C. Decker, R. Russell, and O. Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. https://blockstream.com/eltoo.pdf.

C. Burchert, C. Decker, and R. Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file//49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.

S. Dziembowski, S. Faust, and K. Hostakova. General State Channel Networks. ACM CCS 2018.

a19q3. Lerna claim-UTxO implementation repository. Commit 66ff116, consulted 6 June 2026. https://github.com/a19q3/Lerna.